❸ Constrains the isbn field to be unique

When you let Flyway manage changes to the database schema, you gain all the benefits of version control. You can now start a new PostgreSQL container following the instructions provided in section 5.1.2 (if you still have the previous one running, remove it with `docker rm -fv polar-postgres`), run the application (`./gradlew bootRun`), and verify that everything works correctly.

NOTE In the repository accompanying the book, you can find useful commands to query the PostgreSQL database directly and verify the schemas and data generated by Flyway (Chapter05/05-end/catalog-service/README.md).

Your autotests will also use Flyway. Go ahead and run them; they should all succeed. Once you're done, push your changes to your remote Git repository, and check the commit stage results from GitHub Actions. They should be successful as well. Finally, stop the application execution (Ctrl-C) and the PostgreSQL container (`docker rm -fv polar-postgres`).

In the final section, you'll learn how to use Flyway migrations to evolve a database.

5.4.3 Evolving a database with Flyway

Imagine that you completed the Catalog Service application and deployed it to production. Employees of the bookshop have started adding books to the catalog and gathered feedback about the application. The result is a new requirement for the catalog's functionality: it should provide information about the book publisher. How can you do that?